

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..25

TROUBLESHOOTING

 Copy as Markdown

Troubleshoot GitHub sign-in

This page helps you identify common Sign in with GitHub failures, explains why they occur, and shows how to resolve them. For underlying issues with the OAuth callback library, see the [flutter_web_auth_2 documentation](#).

Setup checklist

Go through this before investigating a specific error. Most problems come from a missed step.

GitHub Developer Settings

- ☐ Created a **GitHub App** at [Developer Settings](#) (or an **OAuth App** if you specifically need one; OAuth Apps allow only a single Callback URL).
- ☐ Registered every redirect URI you actually use under **Callback URL**: reverse-DNS custom schemes for mobile (e.g., `com.example.yourapp://auth`), and the appropriate web URL: `https://your-domain.com/auth/callback` for Serverpod-hosted Flutter web, or `https://your-domain.com/auth.html` for separately-hosted Flutter web. GitHub Apps accept up to 10 entries; OAuth Apps accept only one.
- ☐ Set **Account permissions** > **Email addresses** to **Read-only** if you need the user's email.
- ☐ Disabled **Active** under the Webhook section, unless you actually use webhooks for non-auth reasons.
- ☐ Set **Where can this GitHub App be installed?** to **Any account** if users outside your account need to sign in.
- ☐ Generated a **Client secret** and stored it (GitHub only shows it once).

Server

- ☐ Added `githubClientId` and `githubClientSecret` to `config/passwords.yaml` under the matching environment (`development:` for local, `production:` for prod), or set the matching



`SERVERPOD_PASSWORD_githubClientId` and `SERVERPOD_PASSWORD_githubClientSecret` environment variables.

- ☐ Added `GitHubIdpConfigFromPasswords()` to `identityProviderBuilders` in `server.dart`.
- ☐ Created a `GitHubIdpEndpoint` file in `lib/src/auth/`.
- ☐ Started the server with `serverpod start`, then created and applied the migration (pressed **M**).

Client

- ☐ Added `client.auth.initializeGitHubSignIn(clientId: ..., redirectUri: ...)` after `client.auth.initialize()` inside `initializeClient()` in your Flutter app's `lib/client.dart`.
- ☐ Both `clientId` and `redirectUri` match values registered on the GitHub App.
- ☐ On **Android**, added the `flutter_web_auth_2 CallbackActivity` to `AndroidManifest.xml` with the **exact** scheme used in your callback URL.
- ☐ On **Web (Serverpod-hosted Flutter)**, registered `FlutterWebAuth2CallbackRoute` via `pod.webServer.addRoute(...)` in `server.dart` before `pod.start()`. On **Web (separately-hosted Flutter)**, created `web/auth.html` in your Flutter project. See [Web](#) for both flows.
- ☐ On **web**, ran Flutter on a fixed `--web-port` matching the port registered in the GitHub App's callback URL.

Sign-in fails with redirect_uri_mismatch

Problem: The OAuth flow fails with a `redirect_uri_mismatch` error, or GitHub shows a "The redirect_uri MUST match the registered callback URL for this application" error page.

Cause: The `redirectUri` your Flutter app sent to GitHub does not exactly match any of the **Callback URL** entries on your GitHub App.

Resolution: Open your GitHub App's settings and verify the **Callback URL** entries match your app's `redirectUri` exactly. The match is strict: scheme, host, port, path, casing, and trailing slashes all count.

Common mistakes:

- Trailing slashes (`https://your-domain.com/auth/callback/`) or port differences.
- Wrong scheme (`http` vs `https`, or mismatched custom scheme like `myapp:` vs `MyApp:`).
- For Serverpod-hosted Flutter web, forgetting to register `FlutterWebAuth2CallbackRoute` via `pod.webServer.addRoute(...)` so the `/auth/callback` route is never registered. Hitting the URL returns 404.
- For separately-hosted Flutter web, missing the `auth.html` file in the Flutter project's `web/` folder.
- Flutter dev server running on a random port. Pass `--web-port=<port>` to `flutter run` so the origin is stable across restarts.

Callback never returns to the Flutter app

Problem: The user authorizes the app on GitHub successfully, but the Flutter app never receives the result. The browser sits on a blank page or the OAuth window hangs.

Cause: The browser was redirected to a URL that does not serve the callback page (web), or the callback custom scheme is not registered with the platform (mobile).

Resolution:

- **Web (Serverpod-hosted Flutter):** Confirm `FlutterWebAuth2CallbackRoute` is registered on `pod.webServer` and the provider redirects to that route's URL.
- **Web (separately-hosted Flutter):** Confirm `web/auth.html` exists in your Flutter project and contains the callback script from [Web callback page \(auth.html\)](#).
- **Android:** Verify the `<data android:scheme="..." />` value in `AndroidManifest.xml` matches the scheme in your callback URL exactly.
- **iOS / macOS:** Universal Links require HTTPS callback URLs and associated-domain entitlements. Standard custom-scheme callbacks work without extra configuration.

Sign-in succeeds but the user has no email

Problem: The user signs in successfully in the app, but the server-side `GitHubAccountDetails.email` value is `null`.

Cause: GitHub users can keep their email private, and the OAuth response will return `null` for `email` in that case. Your app may have a custom validator that rejects accounts without an email and blocks the sign-in.

Resolution:

- If you do not strictly need an email, relax your validator. The default validator only checks that `userIdentifier` is non-empty, which works for private-email users. See [Custom account validation](#).
- If you do need an email, confirm your GitHub App requests the **Account permissions > Email addresses** permission as **Read-only**. Even with permission, GitHub returns `null` if the user does not have a verified primary email.
- Surface a helpful message to the user instead of failing silently.

clientId or redirectUri missing at initialization

Problem: The app throws on startup with an error about `clientId` or `redirectUri` being missing or empty.

Cause: Unlike Google or Apple, GitHub does not have native platform-specific config files. The `clientId` and `redirectUri` must be passed explicitly to `initializeGitHubSignIn` (or read from `--dart-define`).

Resolution: Either pass the values directly:

```
await client.auth.initializeGitHubSignIn(  
  clientId: 'your-github-client-id',  
  redirectUri: 'myapp://auth',  
);
```

Or read them from `--dart-define`:

```
flutter run \  
  --dart-define=GITHUB_CLIENT_ID=your-github-client-id \  
  --dart-define=GITHUB_REDIRECT_URI=myapp://auth
```

See [Configuring client IDs on the app](#).

Sign-in works in dev but fails after deploy

Problem: GitHub sign-in works locally but fails in production with `redirect_uri_mismatch` or a similar error.

Cause: The production callback URL is not registered on the GitHub App, or the production Flutter build is using the dev `redirectUri`.

Resolution:

1. Open your GitHub App's settings and confirm the production callback URL is listed under **Callback URL** alongside the development one. Both should remain registered so dev and prod work simultaneously.
2. Confirm your production Flutter build is initialized with the production `redirectUri`. The simplest way is to read it from `--dart-define` and pass the production value in your CI/CD or `flutter build` step. See [Publishing to production](#).

Sign-in works for you but not for other users

Problem: Sign-in works for your own GitHub account but other users get an error from GitHub when they try to authorize the app.

Cause: Your GitHub App's **Where can this GitHub App be installed?** is set to **Only on this account**. Users outside your account cannot install or authorize the app.

Resolution: Open your GitHub App's settings and change **Where can this GitHub App be installed?** to **Any account**. The change takes effect immediately.

Organization blocks the OAuth authorization

Problem: A user tries to sign in but sees a GitHub message about the organization restricting access to third-party applications, or the sign-in flow returns with no authorization.

Cause: The user's GitHub organization has **OAuth App access restrictions** enabled, and your app has not been approved for that organization. This is independent of your app's own settings. The organization controls it.

Resolution: The user (or an organization owner) needs to request approval for your GitHub App in the organization's **Settings > Third-party Access** page on GitHub. There is nothing you can do server-side to bypass this. Surface a clear error message to the user explaining the org policy.

Sign-in fails with bad_verification_code

Problem: GitHub returns `bad_verification_code` when your server exchanges the authorization code for a token.

Cause: The authorization code is single-use and short-lived (10 minutes). This error usually means the same code was sent twice (e.g., page refresh during the callback) or the code expired before the server received it.

Resolution:

- Make sure the OAuth callback fires only once. Refreshing the `auth.html` page or navigating back to it after authorization re-sends the now-spent code.
- If the user genuinely took too long to complete sign-in, the code expired. Have them start the flow again from your sign-in button.
- This is a transient error if it only happens occasionally. Investigate the app only if it reproduces consistently.

GitHub API calls from getExtraGitHubInfoCallback fail or rate-limit

Problem: Calls made inside `getExtraGitHubInfoCallback` start returning `401`, `403`, or `X-RateLimit-Remaining: 0`.

Cause: Either the access token has been revoked (the user revoked the app's authorization, or **Expire user authorization tokens** caused it to expire), or you are exceeding GitHub's per-user rate limit (5,000 authenticated requests per hour by default).

Resolution:

- Cache the data you fetch instead of calling the API on every sign-in. The `getExtraGitHubInfoCallback` runs on **every** authentication attempt, so fetching the same data every time burns through rate limits quickly.
- Serverpod does not keep the GitHub access token after sign-in, so fetch what you need inside `getExtraGitHubInfoCallback` and store the results, rather than re-fetching on every sign-in.
- If a single user triggers many sign-ins (e.g., dev iteration), expect to hit the per-user limit. Wait an hour or test with a different account.

Permission changes on the GitHub App do not take effect

Problem: You changed permissions on your GitHub App (for example, added **Email addresses** read access), but existing users still see the old behavior or get errors about missing scopes.

Cause: GitHub does not silently re-grant new permissions to users who authorized the app before the change. Each user must re-authorize the app for the new permissions to apply.

Resolution: Have affected users sign out and sign in again. GitHub will prompt them to approve the updated permissions. For users who never signed in before the change, the new permissions apply immediately.

Server crashes on startup with a missing password

Problem: The server crashes on startup with an error about a missing `githubClientId` or `githubClientSecret` key.

Cause: The keys are missing from `config/passwords.yaml`, are spelled differently, or are not present under the active environment section.

Resolution: Confirm both keys exist under the section matching your run mode (`development:` when running locally with `serverpod start`, `production:` when deployed):

```
development:
  githubClientId: 'your-github-client-id'
  githubClientSecret: 'your-github-client-secret'
```

Quoting the values is a safeguard. YAML parses unquoted values that look like numbers or booleans as those types instead of strings.

Server crashes on first GitHub sign-in with "no such table"

Problem: The server builds and starts, but crashes when a user tries to sign in with GitHub. The error cites a missing table such as `serverpod_auth_idp_github_account`.

Cause: The database migration that creates the provider's tables was never created or applied.

Resolution: In the running `serverpod start` terminal, press **M** to create and apply the migration.

Android sign-in opens GitHub but the callback never fires

Problem: On Android, tapping the GitHub sign-in button opens the GitHub authorization page in a browser, but after authorizing, the browser stays open and the Flutter app never resumes.

Cause: The `CallbackActivity` in `AndroidManifest.xml` is missing, has a wrong scheme, or `android:exported` is not set to `true`.

Resolution: Open `android/app/src/main/AndroidManifest.xml` and confirm the `CallbackActivity` block exists with `android:exported="true"` and the `<data android:scheme="..." />` value matches the scheme in your callback URL exactly. The block is shown in [Android setup](#).

After editing the manifest, run `flutter clean` and rebuild.

Related

- [Setup](#): configure GitHub sign-in on the server and in your app.
- [Customizations](#): configuration options and sign-in UI customization.
- [UI components](#): the sign-in widgets and how to compose them.